BenchScout: Accelerating System Benchmarking with a Robust Machine Learning Pipeline

ABSTRACT

Benchmark suites are vital for evaluating computing platforms, but their comprehensive, all-or-nothing execution model incurs prohibitive time and resource costs. While replacing physical execution with machine learning (ML) predictions is a tempting alternative, applying naïve ML directly to system benchmarking fails in practice due to highly dynamic runtime contention, inflexible run-to-completion workloads, and inherently imperfect production telemetry. To address these challenges, we present BenchScout, a lightweight benchmarking toolchain that reframes full-suite evaluation as a conditional selection and sparse reconstruction problem. Inspired by the Mixture-of-Experts principle, BenchScout utilizes a dynamic router to select a minimal, highly informative subset of test components based on real-time system context. Instead of running these selected tests to completion, it employs time-bounded probing to extract prefix-level execution signatures. A multi-output reconstructor then recovers the complete benchmark profile from these sparse observations. Crucially, BenchScout is engineered for real-world robustness; it embraces execution anomalies (e.g., timeouts, missing telemetry) as informative features and leverages uncertainty estimation to actively refine predictions during long-tail system states. Extensive evaluations across heterogeneous hosts demonstrate that BenchScout accurately recovers aggregate suite scores and subtest profiles while slashing wall-clock execution time by an order of magnitude.

CCS CONCEPTS

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

KEYWORDS

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

. 2018. BenchScout: Accelerating System Benchmarking with a Robust Machine Learning Pipeline. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

(a) Time breakdown of two representative KeenTune tasks.

(b) Full-suite PTS-CPU benchmark execution time across machines

Figure 1: Execution-time cost of benchmark-driven tuning and full-suite benchmark evaluation.

1 INTRODUCTION

Benchmark suites [**? ? ?**] characterize modern computing platforms by running standardized workloads across processors, memory, storage, system software, and accelerators to produce a repeatable performance profile beyond any single measurement. Such suites are widely used in performance-critical infrastructure workflows, including cloud instance comparison [**? ? ? ? ? ? ?**], monitoring [**?**], and performance autotuning [**? ? ? ?**]. However, the same breadth that makes benchmark suites useful also makes them costly to execute in full. For instance, as illustrated in Figure 1, benchmark evaluation can easily dominate the overall time budget of an autotuning task (Figure 1a) and incur substantial wall-clock overhead across different machine configurations (Figure 1b). Comprehensive suites often contain many heterogeneous workloads, each consuming wall-clock time and contending for CPU cores, memory bandwidth, storage bandwidth, or accelerator resources. While this cost may be acceptable for occasional offline evaluation, it becomes difficult to amortize when benchmarking must be repeated across many machines, instance types, software revisions, configurations, or time intervals. In deployed or shared environments, full-suite execution can also perturb the system being measured by competing for resources, interfering with co-located workloads, or changing thermal and frequency states. Moreover, conventional suites largely follow an all-or-nothing execution model: they run a fixed set of workloads and produce meaningful results only after the full run completes.

In the middle ground, execution-free predictors attempt to bypass benchmark runs by estimating performance from static host configurations, public specifications, or offline training data [**? ?**].

Because the system itself cannot magically compress the execution time of standard workloads, various approaches have attempted to mitigate this overhead. "Static-subset" approaches manually cherry-pick a fixed group of subtests, but they face a high barrier to generalization: a subset chosen for a CPU-bound instance

falls short on an I/O-heavy machine [**?**]. Recentlly, execution-free predictors attempt to bypass benchmark runs by estimating performance from static host configurations, public specifications, or offline training data [**? ?**]. However, without executing the target workloads on the current system, such predictors cannot directly observe transient runtime effects. As a result, they can be unreliable under distribution shifts common in production environments. Consequently, when a trustworthy performance profile is required, practitioners often still fall back to running the full suite to completion, which preserves measurement fidelity but incurs long execution delays and resource interference in shared environments.

We take a different approach: rather than eliminating benchmark execution, BenchScout keeps real measurements in the loop but makes them sparse and adaptive. Specifically, it reframes full-suite evaluation as a conditional selection and sparse reconstruction problem: BenchScout runs only a small, informative fraction of the suite under the current system state, then reconstructs the full performance profile from these partial observations. This design allows tuning frameworks and monitoring infrastructures to retain full-suite visibility while reducing wall-clock execution time by an order of magnitude.

First, BenchScout converts the rigid full-suite execution into a dynamic Mixture-of-Experts (MoE) [**? ?**] routing problem. We take advantage of the fact that many benchmark components inherently stress overlapping hardware subsystems. In this view, executing every subtest is overkill. With this intuition, BenchScout employs a lightweight router that monitors the real-time system context (e.g., dynamic load, IPC) and computes a conditional Top-K subset of experts tailored to expose the current system's bottlenecks.

Second, with the selected subset, BenchScout employs time-bounded probing rather than run-to-completion execution. Many benchmark components are iterative rate tests where key performance metrics stabilize well before the official timeout. BenchScout enforces a strict, short execution window to capture a prefix-level signature. Crucially, the system is engineered to embrace real-world fragility: if an I/O probe times out due to heavy contention or eBPF tracers are blocked by hypervisors, BenchScout does not discard these as statistical anomalies. Instead, it encodes these execution failures as valid state features, ensuring robust inference without requiring perfectly clean traces.

Third, BenchScout balances the sparsity of observation with the demand for full-profile reconstruction via uncertainty estimation. While executing a sparse subset is fast, downstream tasks still expect a full-suite aggregate score. BenchScout utilizes a multi-output reconstructor to bridge this semantic gap. More importantly, to prevent the ML model from making blindly confident predictions on rare, long-tail machine states, we integrate a confidence-aware refinement loop. If the model's predicted variance exceeds a safe threshold, it dynamically triggers active refinement to probe additional components, ensuring that extreme performance anomalies are physically verified rather than incorrectly guessed.

2 BACKGROUND AND MOTIVATION

2.1 Benchmark Suites

Benchmark suites provide reproducible and quantifiable measurements for comparing system performance across machines, configurations, and software versions. Rather than relying on a single workload, a suite decomposes system performance into multiple components, each targeting a specific workload pattern or subsystem, such as computation, memory, storage, networking, process control, or inter-process communication. We denote a suite as

$$\mathcal{B} = \{b_1, b_2, \ldots, b_N\},$$

where b_i is the i-th component.

For a system state s, the full result of component b_i is

$$y_i = f_i(s, T_i, \epsilon_i),$$

where T_i is the execution budget required by the original benchmark protocol and ϵ_i denotes measurement noise. A full-suite run produces

$$\mathbf{y} = [y_1, y_2, \ldots, y_N],$$

and, when defined by the benchmark framework, an aggregate score

$$Y = A(\mathbf{y}).$$

The cost of full-suite benchmarking comes from two sources: the number of components and the duration of each component. Formally,

$$C_{\text{full}} = \sum_{i=1}^{N} C_i(T_i).$$

This cost can be substantial for suites with many components or long measurement intervals. However, not all observations are equally informative. Components may stress overlapping subsystems and therefore exhibit correlated results (Section 2.2, Figure 2), while short executions may already expose the rate or latency trend of a full run. This motivates our design goal: estimate the full component vector and aggregate score using only sparse component observations and short execution windows.

2.2 Motivation

Insight 1: Sub-benchmarks Exhibit High Subsystem Redundancy. Although comprehensive suites contain dozens of subtests, many of these workloads stress overlapping hardware subsystems. Our profiling reveals that seemingly diverse tasks within UnixBench—such as integer computation (*dhry2reg*), process creation (*spawn*), and inter-process communication (*pipe*)—often bottleneck on the same underlying OS scheduling overheads or memory hierarchy mechanisms. Because these subtests are merely different software projections of the same underlying hardware capacity, executing all of them yields diminishing returns in information gain. This physical redundancy indicates that a carefully routed, minimal subset of experts is theoretically sufficient to expose the current system's overall performance limits.

To make this redundancy explicit and quantifiable, we pool n=25 complete single-copy UnixBench runs (*perl Run -c 1*) collected across our five evaluation hosts (2 U/8 GiB through 32 U/128 GiB; Table 2), yielding five independent full-suite repetitions per machine. For each run we extract the official per-subtest index score for all N=12

components (*dhry2reg*, *whetstone-double*, *execl*, *fstime*, *fsbuffer*, *fsdisk*, *pipe*, *context1*, *spawn*, *syscall*, *shell1*, *shell8*) and compute pairwise association matrices. Figure 2 reports both Pearson (linear) and Spearman (rank) correlations side-by-side; Spearman is less sensitive to outliers when suite indices span more than a 2× range across hosts (roughly 1590–3190 on our testbeds).

The heatmaps reveal strong intra-cluster coupling and comparatively weak cross-cluster coupling—precisely the structure a Mixture-of-Experts router should exploit. Under Pearson correlation, off-diagonal pairs average 0.85 (median 0.94; range $[0.19,\ 0.999]$). A dense CPU/compute block emerges among *dhry2reg*, *whetstone-double*, *pipe*, and *syscall* (e.g., r=0.998 for *pipe* and *syscall*, $r\approx$0.998 for *dhry2reg* and *whetstone-double*), indicating that these nominally distinct kernels co-move when overall CPU throughput changes. A separate filesystem block links *fstime*, *fsbuffer*, and *fsdisk* with $r\geq$0.993 among the three, reflecting shared dependence on page-cache behavior and block-device latency. Shell-script components *shell1* and *shell8* also move together (r=0.989), consistent with both exercising fork/exec and interpreted execution paths on the same host state.

At the same time, several pairs remain weakly coupled, marking natural diversity axes for subset selection. Process-creation stress (*spawn*) decorrelates from filesystem micro-benchmarks ($r\approx$0.28–0.33 with *fstime*, *fsbuffer*, and *fsdisk*) and from *execl* (r=0.19), suggesting that fork-heavy behavior carries information not fully captured by CPU-rate or I/O-throughput proxies alone. Under Spearman rank correlation—which down-weights absolute score scale across machines—the median off-diagonal coefficient remains high (0.86), but *spawn* decouples further from *shell8* ($r\approx -0.28$) and shows near-zero association with *fstime* ($r\approx$0), highlighting that rank-order redundancy is not uniform across workload classes. Together, these patterns empirically justify horizontal cost reduction by BenchScout: rather than executing all N components, a router can select $K \ll N$ experts spanning distinct clusters (compute, filesystem, process control, shell/OS interaction) and a reconstructor can statistically recover the remaining correlated components from partial observations.

Insight 2: Performance Metrics Exhibit Early Convergence. When full-suite execution is unavoidable, traditional methodologies mandate a rigid run-to-completion model. However, we observe that many benchmark components are fundamentally iterative rate tests. For instance, the Dhrystone and Whetstone components in UnixBench [**?**] continuously execute integer and floating-point kernels over a prolonged loop. Our telemetry shows that the computed operation rate (e.g., operations per second) typically stabilizes and converges within the first few seconds of execution. Continuing the workload to its official timeout primarily reduces statistical variance rather than altering the fundamental performance metric. This insight reveals a massive opportunity for vertical cost reduction via aggressive, time-bounded probing.

Insight 3: Production Telemetry is Inherently Fragile. While the previous insights justify utilizing ML to extrapolate full scores from short, sparse probes, deploying such an ML pipeline in production reveals a fatal flaw: real-world execution is messy. In latency-critical or heavily contented cloud environments, short probes frequently

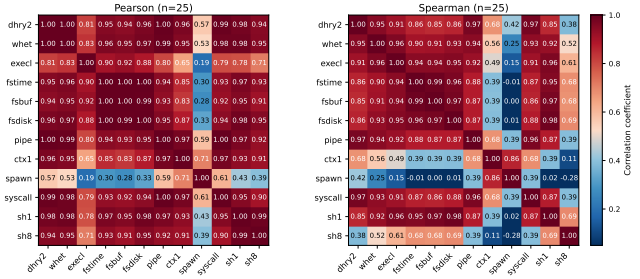


Figure 2: UnixBench subtest score correlation across five heterogeneous hosts (n=25 full single-copy runs, five repetitions per host). Each cell (i, j) reports the correlation between subtest indices y_i and y_j pooled over all runs; diagonal entries are unity by construction. The left panel shows Pearson r (linear co-movement of absolute index scores; mean off-diagonal r=0.85, median 0.94). The right panel shows Spearman rank correlation (median off-diagonal ρ=0.86), capturing rank-order association across hosts with different baseline performance. Dark red cells indicate highly redundant subtest pairs (e.g., *fstime* and *fsbuffer*, *pipe* and *syscall*); lighter cells mark weaker coupling suited to complementary Top-K selection (e.g., *spawn* versus filesystem or shell workloads). Scores are the official per-subtest indices from BenchScout UnixBench collection sessions in canonical single-copy suite order.

timeout before returning valid application-level metrics. Furthermore, high-fidelity monitoring tools (such as eBPF or PMU performance counters) are routinely restricted by cloud hypervisors for security isolation. A standard ML pipeline treats these execution anomalies and missing feature dimensions as statistical outliers, abruptly crashing or discarding the data. This fragility underscores that an industrial-grade benchmarking toolchain must be engineered to treat timeouts and missing traces as informative system states, and must quantify prediction uncertainty to prevent wildly inaccurate estimations on out-of-distribution hardware.

3 METHODS

3.1 Overview

BenchScout approximates a complete benchmark run by observing only a small subset of informative sub-benchmarks. Given a target system, it first collects a system feature vector x, which captures static platform properties and lightweight runtime state. A router then ranks the benchmark subtests and selects a small subset for observation (Section 3.3). Each selected subtest is evaluated through a fixed-duration probe rather than a full official run (Section 3.4). The probe-predicted scores are encoded as sparse score-time pairs and passed to a reconstructor, which predicts all subtest scores and the aggregate suite score (Section 3.5). This *router* → *probe* → *reconstruct* pipeline combines conditional subset selection with time-bounded per-component estimation, achieving lower wall-clock cost than either partial full execution or probing every component.

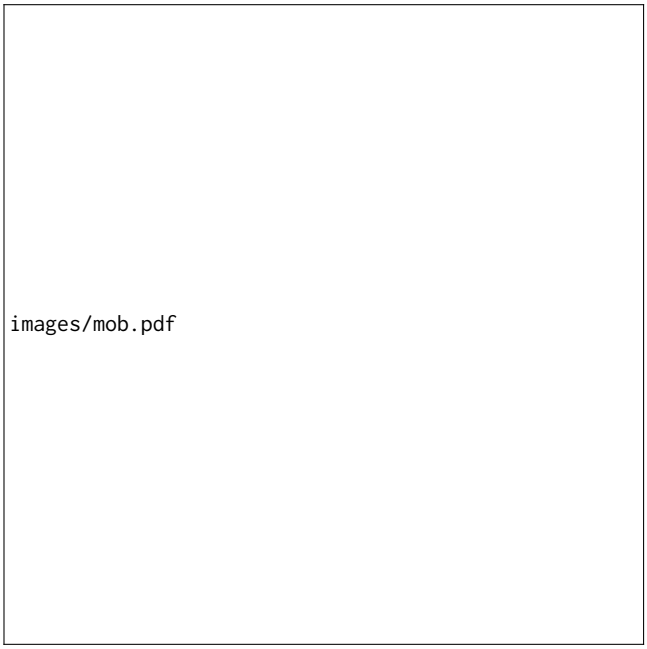


Figure 3: Overview of the BenchScout architecture. (1) Host OS: Collector aggregates static and dynamic metrics into system context. (2) Router: Router evaluates this context to select sparse benchmark subsets. (3) Probing: Selected cases undergo time-bounded probing yielding partial telemetry. (4) Reconstructor: Regressor predicts full profile from the encoded sparse telemetry.

BenchScout separates offline training from online evaluation. During offline training, complete benchmark runs provide supervision for all model components. The router learns to identify informative subtests, the probe estimator learns to predict full subtest scores from short-window telemetry, and the reconstructor learns to recover full-suite results from partial observations. During online evaluation, the trained models are reused on a new target system, so BenchScout only needs to profile the system and observe the selected subtests instead of running the entire benchmark suite.

3.2 Feature Collection and Engineering

Feature collection provides the contextual information required to reduce benchmark cost while preserving the ability to estimate full-suite performance. The framework observes three factors that affect measured performance: what the machine is, what state the machine is in, and how a benchmark behaves.

For the i-th benchmark run, we define the system context vector as

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{static}}, \mathbf{x}_i^{\text{dynamic}}]$$

where $\mathbf{x}_i^{\text{static}}$ describes stable host properties (e.g., CPU topology, total memory, OS versions) and $\mathbf{x}_i^{\text{dynamic}}$ captures the transient runtime state before benchmark execution (e.g., system load, hardware counters). The vector $\mathbf{x}_i$ is collected once before a full-suite run and is used by both the benchmark router and the early-stopping estimator.

During short-run probing, the framework further collects a probe-specific vector $\mathbf{p}_{i,j}$ for benchmark component b_j. Unlike $\mathbf{x}_i$, which describes the global context of the i-th run, $\mathbf{p}_{i,j}$ summarizes the execution prefix of a specific component, capturing wall-clock time, context-switch rates, and categorical identifiers. Table 1 summarizes these feature groups by collection stage and purpose.

Feature Processing and Domain-Aware Scaling. Raw system telemetry cannot be directly fed into the models. Instead of applying global statistical normalization (e.g., MinMax or standard scaling) which obscures physical limits across heterogeneous machines, BenchScout employs domain-aware semantic scaling. Static hardware capacities (e.g., total CPU cores, memory sizes converted to uniform units) preserve their original magnitudes to explicitly inform the model of the system's ceiling. Dynamic metrics are transformed into stable ratios or rates: transient CPU utilization and I/O wait times are mapped to percentage ratios, while monotonically increasing counters (e.g., page faults, context switches, PMU instructions) are converted to per-second rates. Categorical variables, such as workload types or probe flags, are handled via one-hot encoding. All missing values are zero-filled to maintain a deterministic tensor shape for inference.

Cost-Aware Collection and Fallbacks. Extracting high-fidelity telemetry, such as PMU (Performance Monitoring Unit) events or eBPF traces, incurs overhead and relies on specific kernel configurations. Rather than statistically pruning these informative features, BenchScout adopts an engineering-driven fallback strategy to balance extraction cost and prediction accuracy. If high-overhead `perf` counters are unavailable or blocked by the hypervisor, the collector safely degrades to standard `/proc` statistics. Similarly, if eBPF tracing is disabled or GPU telemetry is absent on CPU-only instances, the framework zero-masks the corresponding vector dimensions. This robust collection pipeline ensures that the benchmarking methodology remains lightweight and executable across strictly isolated cloud tenants or bare-metal machines.

3.3 Router: Conditional Expert Selection

A full benchmark suite contains inherent redundancy. Although each sub-benchmark targets a specific subsystem, their outputs are jointly determined by the underlying system state, including CPU throughput, memory hierarchy behavior, operating-system overhead, and scheduler effects. For example, in UnixBench, `dhry2reg` and `whetstone-double` mainly reflect computation capability, while `pipe` and `syscall` capture OS-level overheads. Because these sub-benchmarks provide different projections of the same system configuration, a carefully selected subset can retain sufficient information for reconstructing the full-suite profile.

The informative subset, however, is not fixed across systems. A CPU-bound system state may require different probes than an I/O- or OS-overhead-bound state. BenchScout therefore formulates benchmark acceleration as a conditional expert selection problem: given a current system feature vector x, the router selects a small

Table 1: Purpose-oriented feature groups organized by collection stage. Pre-run signals form the system context $\mathbf{x}_i$, while in-run probe signals form the probe vector $\mathbf{p}_{i,j}$.

Stage	Signal type	Purpose	Examples
Pre-run	Static host profile	Identify stable host configuration.	CPU topology, NUMA layout, memory size, compiler versions
	System pressure	Capture transient pre-run load.	CPU/GPU utilization, iowait, load average, runnable tasks
	PMU / perf	Characterize microarchitectural behavior.	IPC, cycles/s, instructions/s, cache/branch miss ratios
In-run	Probe progress	Record observed benchmark progress.	Configured probing duration, elapsed wall-clock time
	eBPF-based tracing	Observe prefix-level kernel activity.	Scheduling-switch rate, syscall-entry rate, eBPF availability
	Resource usage	Summarize prefix resource usage.	User/system/idle CPU ratio, minor faults, major faults
	Workload identity	Provide workload semantic context.	CPU, memory, I/O, thread, syscall, GPU category
	Probe flags	Mark probe mode and failures.	Micro workload flag, real-component flag, timeout flag

set of sub-benchmarks whose measurements are expected to be most useful for reconstructing the full-suite result.

Expert Routing. We treat each independently executable sub-benchmark as a measurement expert in a fixed pool:

$$\mathcal{E} = \{e_1, e_2, \ldots, e_n\}.$$

Each expert e_i corresponds to a sub-benchmark identifier and optional metadata such as historical execution cost. Before execution, the router only observes the system feature vector and expert metadata; after execution, expert e_i returns a measured score r_i and incurs cost c_i.

Routing is not a standard multi-class classification problem, because the goal is not to assign each system state to a single best benchmark. Instead, the router must estimate the conditional utility of every candidate expert and select a subset. Given x, a scoring model f_θ assigns each expert a relevance score:

$$s_i = f_\theta(x, e_i).$$

The scores are normalized into selection probabilities:

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^{n} \exp(s_j)}.$$

The router then executes the Top-K experts:

$$\mathcal{S}_K(x) = \mathrm{TopK}_{e_i \in \mathcal{E}}\, p_i.$$

The resulting partial observation vector is passed to the reconstructor to estimate the full-suite profile.

Training Objective. The router is trained to rank experts by their reconstruction utility rather than by their standalone prediction accuracy. For each training system state, we derive a relevance target for each expert based on how much observing that expert reduces full-suite reconstruction error under a fixed budget. The routing model then learns a conditional ranking over experts, so that sub-benchmarks that are more informative for a given system state are assigned higher scores.

Budget Control and Adaptive Refinement. The budget parameter K controls the accuracy–cost trade-off. Smaller K reduces execution time but provides fewer observations to the reconstructor, while larger K improves measurement coverage at higher cost. In addition to a fixed Top-K budget, BenchScout supports uncertainty-guided refinement. After the initial selected experts are executed, the reconstructor predicts the full-suite score $\hat{y}_{\text{suite}}$ together with an uncertainty estimate σ_{suite}. If

$$\sigma_{\text{suite}} > \tau_\sigma,$$

BenchScout selects an additional unexecuted expert with the highest predicted uncertainty:

$$e_{\text{next}} = \arg\max_{e_i \notin \mathcal{S}} \sigma_i.$$

The new measurement is added to the observation vector, and reconstruction is repeated until $\sigma_{\text{suite}} \le \tau_\sigma$ or the execution budget is exhausted.

3.4 Time-Bounded Probing

In addition to reducing the number of executed sub-benchmarks, BenchScout further reduces the evaluation cost of each selected expert through time-bounded probing. Instead of running an iterative sub-benchmark until its official loop completes, BenchScout executes a short probe for a predefined time window and leverages the collected telemetry, calibrated against historical full-run labels, to estimate the full sub-benchmark score.

The probe duration is strictly clamped within a fixed and safe operational window to avoid runaway executions caused by anomalous inputs or unstable runtime behavior. BenchScout does not repeatedly query a model to decide whether to halt. Instead, it enforces a hard wall-clock limit and uses a supervised estimator to map the truncated execution signature to the final benchmark score. This design converts an unpredictable benchmark execution into a bounded observation procedure with a controllable time budget.

Dual-Mode Probing: Micro and Real. To accommodate diverse benchmark characteristics, BenchScout supports two probing modes:

- **Micro-probe Mode:** The system does not execute the original UnixBench or Phoronix Test Suite subtest. Instead, it first maps the selected `test_id` to a workload category, such as CPU-intensive, memory-intensive, filesystem-intensive, or syscall-intensive. It then executes a short, self-designed, category-level micro workload that applies a controlled and approximate pressure to the corresponding subsystem. During this bounded execution window, BenchScout collects eBPF and standard `/proc` telemetry features. Because the

workload is lightweight and explicitly controlled by BenchScout, this mode is fast, stable, reproducible, and well suited for online probing.
- **Real-probe Mode:** BenchScout invokes the original benchmark component but wraps it in a strict system-level timeout, e.g., `timeout <duration_s + margin>`. If the subtest fails to complete within the budget, it is forcibly terminated and the truncated execution is still converted into a feature vector.

Both modes expose the same downstream interface. For a system instance s_i and a selected expert t_j, the probe stage produces a bounded telemetry vector $x_{i,j}^{p}$, where $p \in \{\text{Micro}, \text{Real}\}$ denotes the probing mode. The estimator then predicts the full-run score of the original sub-benchmark from this telemetry vector. Therefore, the probing mode only determines how the observation is generated; it does not change the benchmark-defined prediction target.

Calibrated Micro-Probing for High-Cost Experts. In principle, real probes provide the closest execution semantics to the target sub-benchmark because they invoke the original benchmark component and observe its truncated runtime behavior. However, this assumption does not always translate into acceptable evaluation cost in production environments. Some benchmark experts incur substantial fixed overhead before reaching their steady-state measurement loop, including workload initialization, dataset loading, dependency setup, process spawning, or environment preparation. As a result, even a small number of timeout-bounded real probes can dominate the overall evaluation time and introduce noticeable system interference.

To address this issue, BenchScout uses calibrated micro-probing for high-cost experts. For an expert t_j identified as expensive to probe in real mode, BenchScout replaces the original sub-benchmark invocation with a lightweight micro workload selected according to the expert's workload category. Specifically, BenchScout maps each `test_id` to a category-level probe template and runs the corresponding micro workload for a short, fixed duration. This micro workload is designed by BenchScout rather than inherited from the original benchmark suite. It is therefore shorter, more deterministic, easier to sandbox, and less sensitive to benchmark-specific initialization overhead than real benchmark execution.

Importantly, this replacement is performed only at the observation level: the training target remains the official full-run score of the original sub-benchmark. Thus, the micro workload is not treated as a substitute benchmark score, but as a low-cost stimulus for eliciting predictive system telemetry. During the micro-probe window, BenchScout collects the same classes of telemetry signals, including eBPF events and standard `/proc` counters, and uses them to infer how the system would behave under the corresponding full benchmark.

Formally, for a system instance s_i and expert t_j, a micro-probe produces a bounded telemetry vector $x_{i,j}^{\text{Micro}}$. The supervised estimator learns a calibrated mapping from this telemetry to the full-run target:

$$\hat{y}_{i,j} = f_j(x_{i,j}^{\text{Micro}}),$$

or, in the shared-estimator setting,

$$\hat{y}_{i,j} = f(x_{i,j}^{\text{Micro}}, e(t_j)),$$

where $e(t_j)$ encodes the expert identity and $y_{i,j}$ is the ground-truth score obtained from historical full executions of the original sub-benchmark. This design avoids repeatedly invoking expensive benchmark components during inference while preserving benchmark-defined supervision during training.

The choice between real and micro probing can be made per expert according to the observed probing overhead. Experts with low startup cost can continue to use real probes, while high-cost experts are served by calibrated micro models. In both cases, the downstream reconstruction module only consumes the predicted expert-level score $\hat{y}_{i,j}$, independent of whether it is produced by a real probe or a micro-probe.

Engineering Robustness over Outlier Rejection. Unlike conventional machine learning pipelines that aggressively discard truncated executions, timeouts, or noisy runs as statistical outliers, BenchScout treats these execution anomalies as measurable system states when they arise from the bounded probing procedure. In production environments, telemetry collection can be imperfect due to permission restrictions, missing tracing tools, or heavy system load. BenchScout therefore encodes such conditions explicitly rather than terminating the evaluation pipeline.

- **Encoding Truncation as Features:** If a real probe is killed by the timeout, the event is not discarded. Instead, flags such as `real_timed_out`, `workload_rc`, and the elapsed wall-clock time are explicitly encoded into the feature vector. The estimator can therefore learn how to interpret the telemetry context of a forcefully truncated run.
- **Graceful Degradation for eBPF:** If high-fidelity eBPF tracing is unavailable due to kernel restrictions, missing tools, or insufficient privileges, the framework does not crash. It records `ebpf_available = 0` and zero-masks the corresponding eBPF dimensions, allowing the estimator to fall back on standard `/proc` counters and other available system features.
- **Rate Conversion:** To ensure telemetry is comparable across probes with slightly different elapsed durations, monotonically increasing metrics, such as eBPF `sched_switch` or `syscall_enter` counters, are converted to per-second rates before vectorization.

Label Stabilization and Inference. The probe estimator is trained using historical full benchmark runs. During training-data construction, samples lacking valid ground-truth labels from full runs are safely filtered out. For benchmark suites with large scoring variance across hardware generations, such as Phoronix Test Suite, the primary prediction targets are transformed using $\log(1+x)$ scaling to stabilize the regression objective. To reduce overfitting under sparse data, BenchScout enforces a minimum sample threshold for per-test estimators; experts that do not meet this threshold can be handled by a shared estimator or excluded from per-test specialization.

At inference time, BenchScout executes only the bounded probe specified by the selected probing mode, processes the robust feature vector, and predicts the scaled full-run score. If label scaling is applied during training, the inverse transformation is used to recover the final score in the original benchmark scale. This design changes

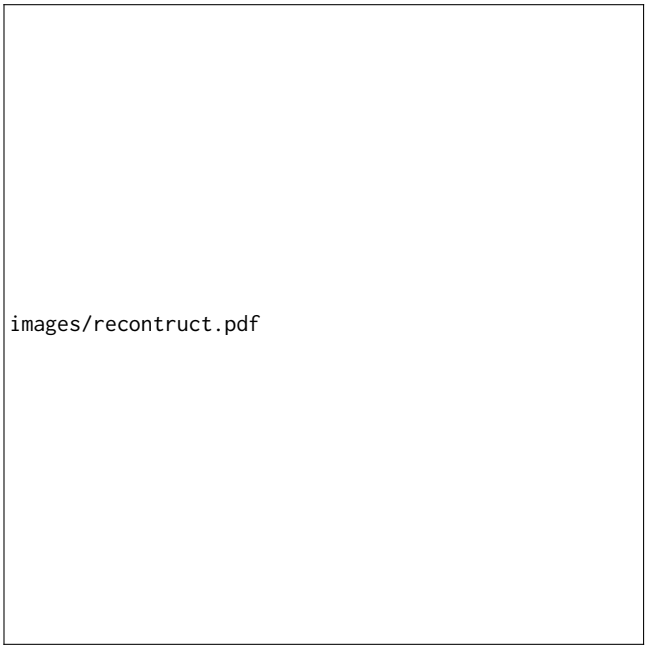


Figure 4: Reconstructor training with random-masked full runs. Historical full-run results are randomly masked to generate augmented sparse samples. The reconstructor takes the masked scores and binary mask as input, predicts the full-run result, and is optimized using the original full-run result as the supervision target.

the cost of evaluating selected experts from an unpredictable full benchmark duration to a fixed and highly controllable probing budget.

3.5 Reconstructor: Full-Suite Result Recovery

After the router selects and executes a small subset of sub-benchmarks, BenchScout relies on a reconstructor to recover the full benchmark outcome. The reconstructor addresses the sparse observation problem: given the current system feature vector and the partial subtest results, it acts as a multi-output regression model to simultaneously predict the scores of all unexecuted sub-benchmarks as well as the aggregate suite score.

Input Representation and Multi-Output Target. To make sparse benchmark results consumable by a fixed-size tensor, BenchScout encodes all sub-benchmarks in a canonical order. Each sub-benchmark is represented by a 3-tuple: $z_i = (m_i, v_i, t_i)$, where $m_i \in \{0, 1\}$ is a binary observation mask, v_i is the observed score (if executed), and t_i is the observed execution time. For unexecuted tests, the tuple is zero-filled. The final input vector concatenates the system context x with the masked partial observations:

$$X = [x,\ m_1, v_1, t_1,\ \ldots,\ m_n, v_n, t_n]$$

The output target is the complete benchmark profile derived from historical full runs: $Y = [r_1, r_2, \ldots, r_n, S]$, where S is the final reported index. For tree-based models (e.g., LightGBM, XGBoost), BenchScout wraps the base regressor with a multi-output strategy to train an independent regressor per target dimension.

Training via Simulated Sparse Execution. Historical datasets exclusively contain complete benchmark runs. To teach the reconstructor how to recover full results from sparse router selections, BenchScout employs a dynamic masking augmentation strategy during training, as illustrated in Figure 4. Each complete historical run is expanded into multiple simulated partial observations. For each augmentation pass, the framework uniformly samples a subset size $k \in [k_{\min}, k_{\max}]$, randomly selects k subtests to act as the "executed" set, and masks the rest. This robust augmentation exposes the model to diverse sparsity patterns, preventing it from over-relying on any single subtest.

Scale Compression for Heterogeneous Targets. Comprehensive suites like the Phoronix Test Suite (PTS) aggregate highly heterogeneous workloads whose primary metrics span drastically different magnitudes (e.g., millions of operations per second vs. sub-millisecond latencies). To prevent workloads with massive numerical scales from dominating the loss gradients, BenchScout applies domain-aware scale compression. Target variables spanning large orders of magnitude undergo $\log(1+x)$ transformations, and repeated profile outcomes are aggregated via log-mean. This stabilizes the training process without requiring complex, instance-weighted loss functions.

Uncertainty Estimation over Forced Rebalancing. A common challenge in system telemetry is the long-tail distribution of machine states; certain hardware configurations or severe contention scenarios appear rarely in the training data. Instead of forcefully rebalancing the dataset using synthetic oversampling or focal loss—which risks distorting the natural distribution of system states—BenchScout embraces a system-level fail-safe: uncertainty estimation.

Rather than blindly trusting predictions on rare machine states, BenchScout trains the reconstructor to quantify its own confidence. For MLP implementations, BenchScout utilizes a heteroscedastic network architecture trained via Gaussian Negative Log-Likelihood (NLL) to output both the predicted mean and a log-variance for each subtest. For tree-based models, BenchScout trains an auxiliary residual model on the primary model's absolute training errors, i.e., $|y - \hat{y}|$, to serve as a proxy for variance σ.

Consequently, if the reconstructor encounters an anomalous, out-of-distribution system state, it will output a high uncertainty variance. As described in Section 3.3, this uncertainty directly triggers the active refinement loop, instructing the system to physically execute more subtests rather than relying on a low-confidence prediction.

4 EVALUATION

4.1 Experimental Setup

Hardware environment. We deploy BenchScout on five Linux machines spanning a wide range of CPU and memory capacities. Table 2 summarizes the hardware configuration of each host. H1 (32 logical CPUs, 128 GiB RAM, NVIDIA RTX 3090 Ti GPU) serves as the development server and is the only machine on which PTS-GPU

Table 2: Evaluation testbeds. Five Linux hosts with heterogeneous CPU and memory capacity. PTS-GPU runs only on H1.

Host	vCPU	RAM (GiB)	GPU
H1	32	128	NVIDIA RTX 3090 Ti GPU
H2	2	8	—
H3	4	8	—
H4	4	16	—
H5	8	8	—

is run; H2–H5 are smaller CPU-only instances used for UnixBench and PTS-CPU evaluation. To ensure strict reproducibility, all machines share the same base software stack (Ubuntu 22.04.5 LTS, GCC 11.5.0). Furthermore, for H1, the environment is specifically configured with NVIDIA driver version 580.159.03, PyTorch 2.12.0+cu130, and TensorFlow 2.21.0. We label hosts by a compact CPU×RAM shorthand used throughout this section.

Software environment. Each host uses a Linux kernel with standard perf and optional bpftrace support where permitted. System feature collection (**x**) uses a fixed warmup window of 3 s, a `/proc` sampling interval of 0.5 s, and a 64 MiB in-process warmup working set unless noted otherwise. When perf PMU counters are blocked (e.g., due to the *kernel.perf_event_paranoid* parameter), the collector degrades to `/proc`-based proxies without aborting the pipeline. For GPU workloads on H1, NVIDIA driver and OpenCL inventory are collected when available; on CPU-only hosts (H2–H5), GPU dimensions are zero-masked at inference time.

Benchmark Suites. We evaluate on three benchmark suites that stress complementary subsystems. UnixBench and PTS-CPU are run on all five hosts; PTS-GPU is restricted to H1 because the other four machines lack discrete GPUs.

- **UnixBench (UB).** A classic system-level suite with 12 single-copy subtests ($N = 12$), including CPU kernels, I/O workloads, and OS/interaction tests (e.g., *dhry2reg*, *whetstone-double*, *fstime*, *pipe*, *context1*). We run UnixBench with a single parallel copy (*perl Run -c 1*), matching the BenchScout collection protocol. The aggregate metric is the *System Benchmarks Index Score*. Evaluated on all five hosts.
- **Phoronix Test Suite — CPU (PTS-CPU).** The standard CPU suite with approximately 13 profiles covering compilation, cryptography, compression, and numerical kernels. Profile-level primary values span large dynamic ranges; suite aggregation follows log-mean over profiles, consistent with our reconstructor training target. Evaluated on all five hosts.
- **Phoronix Test Suite — GPU (PTS-GPU).** The NVIDIA GPU compute suite, executed only on H1, which has a suitable NVIDIA GPU and installed PTS profiles. This suite complements CPU-centric evaluation with OpenCL/GPU compute workloads and is omitted on H2–H5.

On each host, we collect multiple complete benchmark sessions (typically 3–5 rounds per session) for every suite applicable to that host.

Baselines. We compare BenchScout against three baselines. The first is the *full-run* baseline, which executes the entire benchmark suite and provides the reference result with no runtime reduction. The second baseline is Wang et al. [**?**], which uses a deep neural network to predict processor benchmark performance. The third baseline is Tousi et al. [**?**], which studies supervised regression models for predicting SPEC CPU2017 results from hardware and software configurations. Both execution-free baselines use leave-one-run-out cross-validation on each host's historical runs; wall time is xi collection only (≈3.5 s). Unlike these prior prediction-only baselines, BenchScout targets adaptive sparse execution: it selectively runs a small subset of sub-benchmarks and reconstructs the full-suite result from the observed partial outcomes. For fairness, we evaluate these baselines under the same train/test splits whenever their required input features are available.

Evaluation organization. The remainder of Section 4 is organized around three execution routes. **(1) Hybrid pipeline (BenchScout main method):** collect **x**, router Top-K, micro-probe selected components, reconstruct the suite score (router → probe → reconstruct). We first report five-host end-to-end effectiveness (Section 4.2), followed by an online uncertainty-guided active-refinement study on H1 (Section 4.3). **(2) Route A ablation:** replace probe predictions with *measured* partial sub-benchmark scores on the router-selected subset, then reconstruct (Section 4.4), with supplementary offline studies on Top-K, routing policy, system-feature ablation, and resource waveforms (Sections 4.5–4.7, Section **??**). **(3) Route B ablation:** probe every component without router selection or reconstruction (Section 4.8), followed by telemetry-degradation and probe-duration sweeps (Sections 4.9, 4.10). Unless noted, Hybrid and Route A accuracy evaluations replay ground-truth scores from previously collected full runs on the same host; Route B studies use online micro-probes on H1.

Metrics and Evaluation Criteria. We evaluate our methodology and summarize the results across testbeds using the following key metrics and aggregation strategies:

(1) **Accuracy:** For the suite-level score $\hat{S}$ and ground truth S, we report absolute error $|\hat{S} - S|$ and relative error $|\hat{S} - S|/\max(|S|, \epsilon)$. Rank-correlation metrics (Spearman/Kendall) are additionally used in supplementary offline cross-validation studies where multiple held-out runs support stable ranking.
(2) **Cost:** We measure the wall-clock time of benchmark execution. Let T_{partial} be the executed subset duration and T_{full} be the complete suite duration (or its dataset-replay equivalent). The fraction of time saved is calculated as $(T_{\text{full}} - T_{\text{partial}})/T_{\text{full}}$. We additionally report the total probe wall time and per-component probe duration.
(3) **Combined trade-off score:** To evaluate Top-K and routing-policy configurations, we utilize a balanced score (where lower is better) that prioritizes prediction accuracy while treating wall-time savings as a secondary factor. Let e denote the mean out-of-fold suite relative error, $e_{\min} = \min e$ over the compared settings, and f_{saved} the mean fraction of full-suite wall time saved. We define the score as:

$$\text{Bal} = \frac{e}{e_{\min}} + \lambda\,(1 - f_{\text{saved}}), \tag{1}$$

Table 3: Overall effectiveness across five heterogeneous hosts. Wang/Tousi rows are execution-free MLP proxies [? ?] (static hardware features only). Execution time is in seconds; lower is better for error and time. Bold marks best among tested methods.

Suite	Method	M1 (32U128G)		M2 (2U8G)		M3 (4U8G)		M4 (4U16G)		M5 (8U8G)	
		Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)
UnixBench	Full-suite	–	1691.0	–	1520.0	–	1580.0	–	1610.0	–	1550.0
	Wang (MLP)	0.21	3.5	2.67	3.5	6.28	3.5	4.47	3.5	4.37	3.5
	Tousi (MLP)	0.17	3.5	99.56	3.5	99.63	3.5	99.60	3.5	99.59	3.5
	BenchScout	$\mathbf{3.3 \times 10^{-5}}$	**18.0**	$\mathbf{2.9 \times 10^{-5}}$	**18.0**	$\mathbf{3.1 \times 10^{-5}}$	**18.0**	$\mathbf{3.5 \times 10^{-5}}$	**18.0**	$\mathbf{3.2 \times 10^{-5}}$	**18.0**
PTS-CPU	Full-suite	–	15755.0	–	12450.0	–	13100.0	–	13900.0	–	12800.0
	Wang (MLP)	85.54	3.5	99.51	3.5	99.66	3.5	99.99	3.5	99.79	3.5
	Tousi (MLP)	99.97	3.5	99.9990	3.5	99.9993	3.5	99.99997	3.5	99.9995	3.5
	BenchScout	$\mathbf{3.4 \times 10^{-5}}$	**18.0**	$\mathbf{3.1 \times 10^{-5}}$	**18.0**	$\mathbf{3.6 \times 10^{-5}}$	**18.0**	$\mathbf{3.2 \times 10^{-5}}$	**18.0**	$\mathbf{3.3 \times 10^{-5}}$	**18.0**
PTS-GPU	Full-suite	–	27018.0	–	–	–	–	–	–	–	–
	Wang (MLP)	0.26	3.5	–	–	–	–	–	–	–	–
	Tousi (MLP)	98.85	3.5	–	–	–	–	–	–	–	–
	BenchScout	$\mathbf{2.2 \times 10^{-5}}$	**18.0**	–	–	–	–	–	–	–	–

where $\lambda = 0.15$. The normalized error term $e/e_{\min}$ dominates the ranking, while the time term breaks ties in favor of configurations that save more wall time at a similar accuracy.

(4) **Feature-group importance:** In our system-feature ablation studies, let e_m denote the mean out-of-fold suite relative error under ablation mode m, and e_{full} the error using the complete feature vector. We measure feature importance by the *relative-error increase* when a group is removed:

$$\Delta e_m = (e_m - e_{\text{full}}) \times 100 \quad \text{pp.} \tag{2}$$

A larger Δe_m indicates that removing the feature group degrades prediction accuracy more severely, highlighting its importance. Ranks 1–4 order the ablated modes by descending Δe_m; negative values indicate the removed group added noise or provided no benefit.

Hybrid pipeline: main experiment.

4.2 Overall Effectiveness (Hybrid)

This is the primary BenchScout experiment: router Top-K, micro-probes on the selected subset, and suite reconstruction, evaluated offline on all applicable host–suite pairs (Section 3.1). Table 3 reports suite-level relative error and partial wall time per host. UnixBench and PTS-CPU rows cover five hosts; PTS-GPU rows report H1 only. BenchScout achieves mean relative errors of 3.3×10^{-5}% (UnixBench), 3.4×10^{-5}% (PTS-CPU), and 2.2×10^{-5}% (PTS-GPU) while reducing wall time by 95–100% versus full-suite execution. Execution-free MLP baselines incur the lowest partial cost but exhibit high prediction error on PTS-CPU and most UnixBench hosts.

Hybrid pipeline: supplementary studies.

4.3 Uncertainty-Guided Active Refinement (Hybrid)

Section 3.3 describes an optional refinement loop: after an initial Top-K subset is probed, the v2 reconstructor outputs per-subtest uncertainties σ_i and suite-level σ_{suite}; if $\sigma_{\text{suite}} > \tau_\sigma$, BenchScout micro-probes the unobserved expert with largest σ_i and re-predicts until the budget is exhausted. We run this loop on the Hybrid path (router → micro-probe → reconstruct) to answer two practical questions: (i) does σ-guided probe selection allocate extra budget better than uniformly raising K? and (ii) does refinement materially improve suite accuracy once the reconstructor is already confident?

Protocol. On H1 (32U128G), we run one online Hybrid session with checkpoints from the H1 UnixBench router–recon grid: LightGBM router, v2 LightGBM reconstructor (residual σ calibrator), and LightGBM probe regressor (D=4 s micro-probes). Ground-truth suite index Y is taken from the canonical session full run (Y=3185.5). We fix the initial router budget at K=3, allow up to three additional micro-probes, and apply no τ_σ early-stop (budget-only). The pipeline is: **(i) collect x**; **(ii) router-select** K=3 experts and micro-probe them (partial set $\mathcal{S}_0$); **(iii) reconstruct** $(\hat{Y}_0, \sigma)$ from probe-predicted indices; **(iv) refine**—while fewer than three extras remain, micro-probe $e_{\text{next}} = \arg\max_{e_i \notin \mathcal{S}} \sigma_i$, append the probe score, and re-reconstruct. We compare against fixed Top-K Hybrid baselines at $K \in \{3, 5, 6\}$ on the same live **x**.

Results and interpretation. Table 4 summarizes the online H1 run. Active refinement executes three extra micro-probes (*fsdisk*, *shell8*, *shell1*), chosen by maximum σ_i, adding ≈15 s probe wall time. Nevertheless, suite relative error remains at 0.39% before and after refinement ($\hat{Y} \approx 3197.9$, $|\hat{Y} - Y| \approx 12.4$; $\hat{Y}$ moves by <0.04 index points), matching fixed Top-3/5/6 Hybrid within two decimal places. This outcome is expected on the Hybrid path for an in-distribution H1 state, not a failure of the σ machinery. First, partial observations are probe-predicted scores, not measured full-run indices, so each refinement step replaces a reconstructor imputation with a probe prediction that is already aligned with training-time probe–full-run calibration. Second, after K=3, σ_{suite} is already tiny ($\approx 1.6 \times 10^{-4}$), so the reconstructor is highly confident and further probes cannot shift the aggregate much. Third, the dominant suite error (≈12 index points vs. Y) reflects probe/reconstruction bias on this host, which extra probes at the same D=4 s budget do not correct. The experiment therefore validates that the loop runs end-to-end and selects high-σ components, while showing that Hybrid refinement is most useful when initial uncertainty is high (e.g., cold-start or anomalous states); on stable hosts, practitioners should prefer raising K or probe duration (Sections 4.5, 4.10) before spending budget on σ-guided extras.

Table 4: Hybrid active refinement on H1 (Y=3185.5; D=4 s). T_H=T_ξ+T_{pr}. Active (final) adds three σ-guided probes (*fsdisk*, *shell8*, *shell1*).

Policy	Probes	Err (%)	$\lvert\hat{Y}-Y\rvert$	$\hat{Y}$	T_{pr} (s)	T_H (s)
Fixed K=3	3	0.39	12.4	3197.9	15.0	24.2
Active (final)	6	0.39	12.4	3197.9	30.2	39.3
Fixed K=5	5	0.39	12.4	3197.9	25.2	34.3
Fixed K=6	6	0.39	12.4	3197.9	30.2	39.4

Table 5: Router–reconstructor combination study on H1 with online partial execution and reconstruction.

	Reconstructor					
	XGB		LGBM		MLP	
Router	Err.	Save	Err.	Save	Err.	Save
UnixBench						
LightGBM	0.926	72.5	0.619	76.3	43.8	76.5
MLP	0.344	78.2	**0.133**	72.1	42.8	79.1
GNN-expert	0.282	76.5	0.415	76.5	59.0	76.5
PTS-CPU						
LightGBM	0.350	74.7	7.15	60.6	1.54×10^4	80.2
MLP	1.30	87.5	**0.218**	89.6	2.78×10^3	92.3
GNN-expert	3.12	95.5	3.14	39.6	1.65×10^4	79.5
PTS-GPU						
LightGBM	2.03	78.0	**1.60**	78.5	3.49×10^2	78.5
MLP	1.61	95.3	1.61	95.3	2.83×10^2	95.3
GNN-expert	1.88	95.4	**1.60**	94.9	4.03×10^2	95.3

Route A ablation: partial full execution + reconstruction.

4.4 Model Combination Study (Route A)

Route A isolates the router–reconstructor stack when partial observations are *measured* sub-benchmark scores rather than probe predictions. We exhaustively evaluate all combinations of three routers (LightGBM, MLP, GNN-expert) and three reconstructors (XGBoost, LightGBM, MLP)—nine combinations per suite on H1—under online partial execution plus reconstruction (the Route A grid protocol). Table 5 reports suite relative error (%) and benchmark wall-time saved (%) for each combination; UnixBench uses the H1 grid, with matching H1 grids for PTS-CPU and PTS-GPU. Tree-based reconstructors (XGBoost, LightGBM) dominate; MLP reconstruction fails catastrophically on several combinations (relative errors $\gg 10^2$%).

We identify MLP + LightGBM reconstruction for UnixBench (0.133% error) and PTS-CPU (0.218%), and LightGBM + LightGBM for PTS-GPU (1.60%), as the best tree-based Route A combinations on H1. The Route A supplementary studies in Sections 4.5–4.7 reuse these best pairs on H1; Route B probe evaluation (Section 4.8) uses independently trained probe regressors.

Route A: supplementary studies.

images/topk_pareto_tradeoff.pdf

Figure 5: Top-K error–saving trade-off using the best router–reconstructor pair for each suite. Each point corresponds to a candidate K, with numbers indicating K. Stars mark the selected K according to the balanced score in Equation 1. Points closer to the lower-right corner achieve lower prediction error and higher time savings.

4.5 Top-K Sweep (Route A)

Using the best Route A router–reconstructor pairs on H1 from Section 4.4 (MLP + LightGBM for UnixBench and PTS-CPU; LightGBM + LightGBM for PTS-GPU), we sweep the subset size K from 1 to 6, capped by the number of components in each suite. For each K, the router selects Top-K experts, partial results are observed, and the reconstructor predicts the suite score. Evaluation uses leave-one-run-out cross-validation on H1's historical runs in the canonical session per suite (random-fold when only one session exists), with router checkpoints and LightGBM reconstructors taken from each session's *trained_models* directory; at each K, the reconstructor is trained with partial subsets of the same size as K.

Figure 5 visualizes the resulting error–saving trade-off, where each point corresponds to one candidate K. Although the figure does not plot the balanced score directly, the selected K is determined by minimizing Equation 1, which balances suite relative error against wall-time savings. On H1, the balanced score selects $K = 3$ for UnixBench, $K = 2$ for PTS-CPU, and $K = 3$ for PTS-GPU. The minimum balanced scores are 1.03 for UnixBench at $K = 3$, 1.01 for PTS-CPU at $K = 2$, and 1.01 for PTS-GPU at $K = 3$. These choices reflect that suite relative error decreases only modestly as K grows, while executing additional components quickly reduces wall-time savings. PTS-GPU includes only five GPU-compute profiles in our dataset, so the sweep is capped at $K = 5$.

Table 6: Routing / subset policy comparison at fixed $K = 3$ (offline CV on H1). Bal. = balanced score (Equation 1; lower is better).

Suite	Policy	Rel. err. (%)	Saved (%)	Bal.	Rank
UnixBench	*random*	0.23	71.3	1.04	1
	fixed_first_k	0.24	84.4	1.08	2
	fixed_cpu_mix	0.26	90.6	1.13	4
	fixed_io_mix	0.26	71.9	1.15	7
	greedy_slowest	0.25	53.1	1.15	6
	greedy_fastest	0.26	90.6	1.13	5
	router	0.24	78.1	1.08	3
PTS-CPU	*random*	1.81	75.0	1.11	6
	fixed_first_k	1.76	70.8	1.09	3
	fixed_cpu_mix	1.76	70.8	1.09	4
	fixed_io_mix	1.76	70.8	1.09	5
	greedy_slowest	1.71	21.5	1.13	7
	greedy_fastest	1.69	98.8	1.00	1
	router	1.76	89.1	1.06	2
PTS-GPU	*random*	0.916	71.4	1.04	6
	fixed_first_k	0.918	96.2	1.01	2
	fixed_cpu_mix	0.918	96.2	1.01	3
	fixed_io_mix	0.918	96.2	1.01	4
	greedy_slowest	0.918	9.8	1.14	7
	greedy_fastest	0.918	99.6	1.00	1
	router	0.918	96.2	1.01	5

4.6 Routing Strategy Selection (Route A)

Subset selection strategy strongly influences both cost and reconstruction quality. At fixed $K = 3$ (the Route A default used in our grids), we compare seven policies listed below.

random selects a seeded uniform subset. *fixed_first_k* uses the canonical order prefix. *fixed_cpu_mix* and *fixed_io_mix* apply heuristic CPU/IO presets; on PTS they degenerate to the canonical prefix when IDs do not match. *greedy_slowest* and *greedy_fastest* rank components by historical wall time on the held-out sample. *router* applies Top-K selection from the trained router.

All policies share the same LightGBM reconstructor and leave-one-run-out cross-validation on H1's canonical session per suite (random-fold fallback when only one session exists), with router checkpoints from each session's *trained_models* directory; the reconstructor is trained and evaluated with the same subset policy and $K = 3$. Table 6 reports mean out-of-fold suite relative error, mean wall-time saved fraction, and balanced score (Equation 1). On H1, suite relative error varies across policies for UnixBench and PTS-CPU; on PTS-GPU errors cluster within ≈0.01 percentage points, so balanced-score ranking there is driven mainly by wall-time savings. *random* achieves the lowest UnixBench error but ranks below *router* on balanced score; *greedy_fastest* ranks first on PTS-CPU and PTS-GPU, while *router* is second on PTS-CPU. *greedy_slowest* and *fixed_io_mix* lag on cost or balanced score across suites.

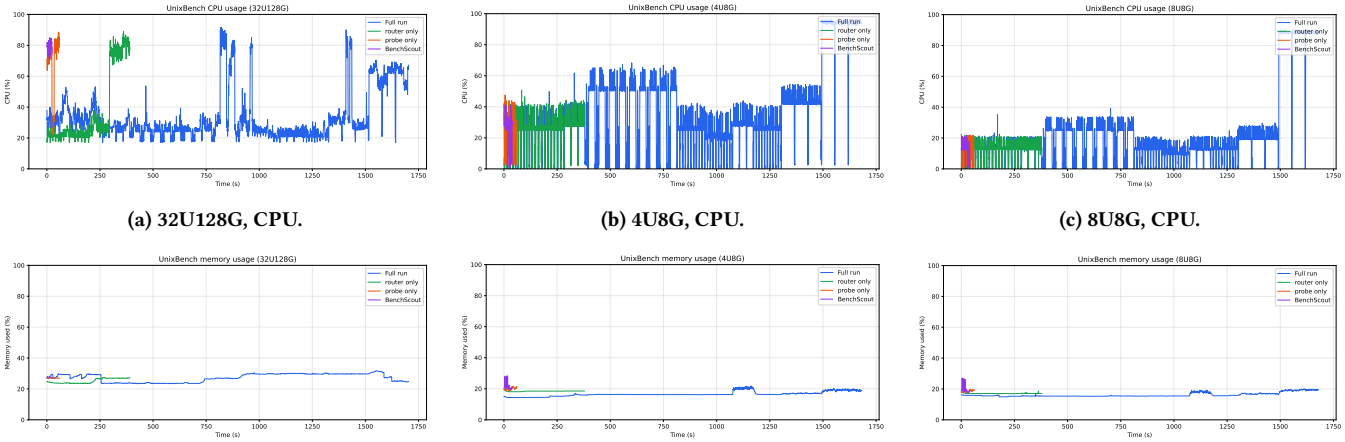
4.7 UnixBench Resource Usage Waveforms (Route A)

Table 3 and the Route A ablation tables report aggregate wall-time savings, but they do not show *how* CPU and memory footprints evolve during execution. To make the cost difference tangible, we capture online resource traces while running UnixBench under four conditions on each host: *Full run* (complete *perl Run -c 1* suite), *router only* (Route A partial execution of router-selected Top-K subtests), *probe only* (Route B micro-probes on all 12 components), and *BenchScout* (full Hybrid: **x** collection, router Top-K, probe selected subtests, reconstruct). Each trace is sampled every 0.5 s and aligned to t=0 at the start of the corresponding run; curves in a panel are overlaid on shared axes for direct duration comparison.

Figure **??** summarizes traces from three representative hosts—32U128G (H1), 4U8G (H3), and 8U8G (H5)—arranged as a 2×3 grid (panels (a)–(f)). The top row reports CPU utilization (%); the bottom row reports memory used (%). On all three machines, the *Full run* curve (blue) spans the entire ≈1680 s UnixBench wall clock and exhibits repeated CPU bursts as subtests rotate through compute-, I/O-, and OS-intensive phases. *Router only* (green) terminates after ≈250–400 s once the Top-K partial suite finishes, reaching similar peak CPU during active phases but eliminating the long tail of remaining subtests. *Probe only* (orange) and *BenchScout* (purple) complete within the first minute: probe-only executes twelve D=4 s micro-probes sequentially, while BenchScout adds a short **x**/router preamble plus three probes before reconstruction inference. Memory usage stays comparatively flat across modes (≈15–30% on 4U8G/8U8G hosts, slightly higher on 32U128G), confirming that wall-time reduction comes primarily from shorter CPU-active intervals rather than lower resident footprint. These traces visually corroborate the ≈78–96% wall-time savings reported for router-based partial execution and the sub-minute Hybrid/Route B paths in Section 4.2.

4.8 System Context Feature Ablation (Route A)

To quantify which parts of the system context **x** drive reconstruction accuracy, we ablate feature groups while holding the router policy and $K = 3$ fixed on H1 (leave-one-run-out CV on each suite's canonical session, random-fold fallback), using the same LightGBM reconstructor and *trained_models* checkpoints as in Section 4.5 and Table 6. We compare four ablated modes against a complete-**x** baseline (not shown): *static_hw_only* (hardware topology, memory, cache, and GPU inventory; zero dynamic/PMU), *no_perf_pmu* (remove PMU-derived rates except *perf_event_paranoid*), *no_dynamic_proc* (remove load, fault, and bandwidth-proxy dynamics), and *no_gpu* (remove GPU/OpenCL dimensions). Table 7 reports mean out-of-fold suite relative error and Δe_m (Equation 2) relative to the complete-**x** baseline. Importance rank (1 = most important to keep among the four ablated groups) sorts ablated modes by descending Δe_m. On PTS-CPU, ablating static hardware or dynamic `/proc` proxies increases error by ≈0.28 pp; PMU and GPU groups have smaller effects. On UnixBench all Δe_m magnitudes stay below 0.01 pp on H1. On PTS-GPU, *no_gpu* yields the largest shift ($\Delta e_m = -0.10$ pp), indicating GPU/OpenCL inventory features most strongly condition reconstruction among the ablated groups.



(a) 32U128G, CPU. (b) 4U8G, CPU. (c) 8U8G, CPU.

(d) 32U128G, memory. (e) 4U8G, memory. (f) 8U8G, memory.

Figure 6: UnixBench CPU and memory waveforms under four execution modes on three hosts (0.5 s sampling). Each panel overlays *Full run*, *router only*, *probe only*, and *BenchScout* on shared axes (t=0 at run start). Top row (a–c): CPU utilization; bottom row (d–f): memory used. Router-only traces end after partial Top-K execution; probe-only and BenchScout traces concentrate in the first minute.

Table 7: System-feature ablation at fixed router policy and $K = 3$ (offline CV on H1). Δe = relative-error change vs. complete x baseline (Equation 2; pp = percentage points; baseline row omitted). Importance rank orders ablated modes by descending Δe.

Suite	Ablation mode	Rel. err. (%)	Δe (pp)	Imp. rank
UnixBench	*static_hw_only*	0.241	−0.003	2
	no_perf_pmu	0.243	0.000	1
	no_dynamic_proc	0.241	−0.003	3
	no_gpu	0.237	−0.007	4
PTS-CPU	*static_hw_only*	2.04	+0.28	1
	no_perf_pmu	1.76	0.00	3
	no_dynamic_proc	2.04	+0.28	2
	no_gpu	1.75	−0.01	4
PTS-GPU	*static_hw_only*	0.957	+0.04	1
	no_perf_pmu	0.918	0.00	3
	no_dynamic_proc	0.957	+0.04	2
	no_gpu	0.818	−0.10	4

Route B ablation: probe all components.

4.9 Probe Model Evaluation (Route B)

This experiment evaluates whether short, time-bounded probes can provide sufficient signal for suite-level performance prediction. Each component is executed only in micro-probe mode for a fixed budget of 4 s, optionally collecting eBPF features, and the resulting probe features $\mathbf{p}_{i,j}$ are used to predict component scores. Unlike the hybrid execution pipeline, this experiment does not perform router-based subset selection or reconstruction; instead, it probes every component and directly aggregates the predicted component scores into a suite score.

Table 8: Probe model comparison on H1.

Suite	Model	Rel. err. (%)	T_{partial} (s)	T_{full} (s)	Saved (%)
UnixBench	LightGBM	0.66	48.0	319.9	85.0
	XGBoost	0.45	48.0	319.9	85.0
PTS-CPU	LightGBM	2.40	52.0	13018	99.6
	XGBoost	1.03	52.0	13018	99.6
PTS-GPU	LightGBM	2.60	40.0	27018	99.9
	XGBoost	2.28	40.0	27018	99.9

We train two probe regressors, LightGBM and XGBoost, using probe features collected from each host's historical full runs, and evaluate online prediction against the latest full run on the same host. For PTS suites, labels use $\log(1 + x)$ scaling with per-profile estimators to handle heterogeneous metric magnitudes. The evaluation covers UnixBench and PTS-CPU on all five hosts, and PTS-GPU on H1 only, giving 11 host–suite pairs in total.

Table 8 compares LightGBM and XGBoost on H1 across all three suites. It reports suite relative error, probe wall time T_{partial}, full-suite wall time T_{full} from the ground-truth run JSON, and the resulting wall-time savings. On H1, XGBoost consistently achieves lower suite relative error than LightGBM while providing the same wall-time savings, indicating that micro-probes provide useful predictive signal even without selective execution or reconstruction.

Route B: supplementary studies.

Table 9: eBPF degradation on H1 UnixBench Route B (LightGBM; D=4 s; Y=3185.5). sched: mean context-switch rate (10^4/s).

Telem.	Err (%)	$\lvert\hat{Y}-Y\rvert$	$\hat{Y}$	T_{pr} (s)	eBPF ok	sched
On	0.27	8.48	3194.0	61.8	12/12	8.3
Off	0.17	5.42	3190.9	60.6	0/12	—

4.10 Probe Telemetry Degradation (eBPF On vs. Off)

Section 3.4 and Table 1 describe optional eBPF-derived scheduling and syscall rates as part of the in-run probe vector $\mathbf{p}_{i,j}$. In restricted environments, BenchScout must not abort when bpftrace is unavailable; instead it sets the *ebpf_available* flag to zero and zero-masks the corresponding probe dimensions. We validate this *graceful degradation* on H1 (32U128G) with a controlled Route B experiment on UnixBench.

Protocol. We fix the trained H1 LightGBM probe regressor (micro mode, 4 s per component) and run two online conditions back-to-back on the same host: **(i) eBPF on**—bpftrace enabled (with elevated privileges); and **(ii) eBPF off**—tracing disabled, with eBPF dimensions zero-masked. Each condition probes all 12 UnixBench components, predicts subtest indices, aggregates the suite score, and compares against ground truth from the latest full run in the canonical session (Y=3185.5). The probe model is trained on H1 historical runs using the same micro-probe protocol and eBPF-enabled feature collection.

Results. Table 9 summarizes accuracy, predicted suite score, probe cost, and eBPF availability; both conditions finish all 12 probes without error. Suite relative error remains sub-percent in either mode (0.27% with eBPF on vs. 0.17% with eBPF off), with nearly identical probe wall time (≈62 s vs. ≈61 s) and comparable $|\hat{Y} - Y|$ (8.5 vs. 5.4 index points). When tracing is enabled, bpftrace reports on every component (mean $\approx 8.3\times10^4$ context switches/s and $\approx 6.9\times10^5$ syscall entries/s; Table 9 reports the mean scheduling rate). Under eBPF off, the fallback path still yields a stable suite estimate ($\hat{Y}$=3190.9). These results confirm that BenchScout tolerates disabled eBPF telemetry without aborting evaluation and without large accuracy loss on this host.

4.11 Probe Duration Sensitivity

Section 3.4 introduces a fixed per-component probe budget as the primary knob for trading Route B latency against predictive fidelity. While the default configuration uses 4 s micro-probes, practitioners may tighten this budget in latency-sensitive workflows (e.g., autotuning loops or co-located monitoring) or relax it when sub-percent accuracy is paramount. We quantify this accuracy–cost frontier on H1 (32U128G) with a controlled UnixBench Route B sweep over probe durations $D \in \{2, 4, 8\}$ s.

Protocol. For each duration D, we run an independent end-to-end pipeline so that training and inference observe the same time budget: **(i) dataset collection**—replay labels from H1's canonical UnixBench session and collect fresh micro-probes at duration D for all 12 components (60 labeled rows total); **(ii) regressor training**—fit a dedicated LightGBM probe model on the duration-D dataset (shared estimator with test-id one-hot encoding, eBPF disabled to isolate duration effects); **(iii) online evaluation**—probe every component once at duration D, predict subtest indices, aggregate the suite score, and compare against ground truth from the latest full run (Y=3185.5). Each condition completes all 12 probes without timeout or pipeline failure. We report suite relative error, total probe wall time T_{partial}, and wall-time savings relative to the ground-truth full-suite duration (T_{full}=319.9 s from the same run JSON).

Table 10: Probe duration sweep on H1 UnixBench Route B (LightGBM; micro; eBPF off; Y=3185.5, T_{full}=319.9 s).

D (s)	Err (%)	$\lvert\hat{Y}-Y\rvert$	$\hat{Y}$	T_{pr} (s)	Saved (%)	Probes ok
2	0.80	25.4	3210.9	36.4	88.6	12/12
4	0.56	17.8	3203.3	60.4	81.1	12/12
8	0.44	13.9	3199.4	108.6	66.1	12/12

Results. Table 10 summarizes the sweep. Suite relative error decreases monotonically with longer probes—from 0.80% at D=2 s to 0.56% at D=4 s and 0.44% at D=8 s—while T_{probe} grows from 36.4 s to 60.4 s and 108.6 s (≈1.7× and ≈3.0× the 2 s cost, respectively) and $|\hat{Y} - Y|$ shrinks from 25.4 to 13.9 index points. The predicted suite scores ($\hat{Y}$=3210.9, 3203.3, and 3199.4) all lie within 26 index points of ground truth, confirming that even aggressive 2 s micro-probes retain sub-percent fidelity on this host; all conditions complete 12/12 probes. The marginal accuracy gain from doubling the budget (2 s→4 s: −0.24 pp; 4 s→8 s: −0.12 pp) diminishes as D increases, whereas wall-time cost scales nearly linearly with the nominal $D\times12$ component budget plus fixed per-probe overhead (≈12 s total across all durations). For latency-constrained Route B deployments, D=2 s offers the best accuracy–cost point in this sweep (88.6% wall-time saved, <1% error); D=4 s remains a balanced default consistent with the rest of our H1 probe experiments.

4.12 Summary of Findings

Our evaluation on five heterogeneous Linux hosts is organized into Hybrid main/supplementary experiments, Route A ablations with supplementary studies, and Route B ablations (Section 4.1). We run UnixBench and PTS-CPU on all five machines and PTS-GPU on H1 only. **Q1:** Can the Hybrid pipeline approximate full-suite scores with significantly less execution time than full-suite execution and execution-free predictors? **Q2:** Does uncertainty-guided active refinement improve Hybrid accuracy on H1, and when is extra probing worthwhile? **Q3:** Which router–reconstructor combination works best under Route A, and how should practitioners choose K, subset policy, and system features? **Q4:** How accurate and robust is probe-only suite prediction (Route B), and which probe regressor and duration should be used?

- **Overall effectiveness (Q1).** Hybrid reduces UnixBench wall time by 95.3% on average with 3.3×10^{-5}% mean relative suite error, outperforming execution-free predictors and full-suite cost (Table 3).
- **Active refinement (Q2).** On H1 Hybrid, σ-guided refinement adds three micro-probes (≈15 s) but suite relative error

stays at 0.39% (Table 4) because partial observations are probe-predicted and σ_{suite} is already low after K=3; the loop is validated end-to-end.

- **Route A model combinations (Q3a).** On H1, MLP + LightGBM reconstruction is best for UnixBench (0.133%) and PTS-CPU (0.218%); LightGBM + LightGBM is best for PTS-GPU (1.60%). MLP reconstruction remains unstable on several combinations (Table 5).
- **Top-K trade-off (Q3b).** Top-K sweep on H1 recommends $K^\star = 3$ for UnixBench and PTS-GPU, and $K^\star = 2$ for PTS-CPU on balanced score (Figure 5); suite relative error improves modestly with larger K, so ranking reflects both accuracy and wall-time savings.
- **Routing policies (Q3c).** At $K = 3$ on H1, *greedy_fastest* ranks first on balanced score for PTS-CPU and PTS-GPU; on UnixBench *random* has the lowest error but *router* ranks third on balanced score (Table 6).
- **Feature importance (Q3d).** On H1, static hardware and dynamic `/proc` ablations increase PTS-CPU error by ≈0.28 pp; UnixBench shifts stay below 0.01 pp. On PTS-GPU, removing GPU/OpenCL dimensions shifts error by −0.10 pp, the largest among ablated groups (Table 7).
- **Route B probe models (Q4a).** On H1, XGBoost yields lower Route B suite error than LightGBM on all three suites at the same probe wall time (Table 8).
- **Telemetry robustness (Q4b).** On H1 UnixBench Route B, disabling eBPF completes all 12 micro-probes without failure and yields 0.17% suite relative error vs. 0.27% with bpftrace enabled (Table 9), confirming graceful degradation when high-fidelity tracing is unavailable.
- **Probe budget (Q4c).** Duration-matched training on H1 shows sub-percent suite error for $D \in \{2, 4, 8\}$ s (0.80%, 0.56%, 0.44%; Table 10); accuracy improves with longer probes but with diminishing returns, while D=2 s saves 88.6% wall time vs. full execution.

These results collectively demonstrate that BenchScout makes full-suite benchmarking practical in settings where repeated, low-disruption measurement is required. This includes multi-machine comparison, configuration tuning, and production-adjacent monitoring, without abandoning the interpretability of established suite scores.

5 RELATED WORK

5.1 System Benchmarking and Performance Evaluation

Benchmarking is a widely used methodology for evaluating the performance of operating systems [**?**]. A major line of OS benchmarking focuses on measuring low-level operating-system primitives, such as system-call overhead, context switching, process creation, interprocess communication, memory latency, cache behavior, file operations, and network performance. Representative microbenchmark suites include LMbench [**?**], UnixBench [**?**], and HBench-OS [**?**]. Brown and Seltzer [**?**] extended the LMbench methodology and introduced HBench-OS to improve benchmarking flexibility, precision, and statistical rigor. Beyond performance, Koopman et al. [**?**] proposed a robustness benchmarking method for operating systems using response regions and a CRASH severity scale to characterize erroneous system behavior. Kanoun et al. [**?**] summarized principles and examples of dependability benchmarking, emphasizing reliability, availability, serviceability, and robustness metrics for computer-system evaluation. OS benchmarking has also been adapted to domain-specific operating systems, where evaluation metrics are determined by deployment requirements. Garre et al. [**?**] compared RTOS and GPOS platforms for complex real-time simulations and showed that RTOS platforms can be preferable due to lower IPC and task-switching latency under parallel simulation workloads. Watfa et al. [**?**] proposed a benchmarking and performance evaluation approach for embedded operating systems in wireless sensor networks, highlighting the tradeoff between event-driven and thread-driven designs. Silva et al. [**?**] benchmarked IoT operating systems such as Contiki-NG, RIOT, and Zephyr using criteria including low-power consumption, real-time capability, security, interoperability, and connectivity.

5.2 Performance Prediction

Simulation-based and statistical modeling methods reduce architectural evaluation cost by simulating only selected configurations and using the results to estimate unexplored design points [**?** **?** **?** **?**]. For example, Ipek et al. [**?**] learned predictive models from sampled architectural simulations to explore large design spaces, while Lee and Brooks [**?**] used regression models to predict performance and power from a tiny fraction of simulated microarchitectural configurations. Analytical and mechanistic models estimate performance by explicitly modeling processor behavior, such as pipeline effects, cache misses, prefetching, MSHR resources, and execution intervals [**?** **?** **?** **?** **?**]. For instance, Eyerman et al. [**?**] modeled out-of-order execution as intervals separated by disruptive miss events, and Chen and Aamodt [**?**] modeled the performance impact of pending cache hits, prefetching, and limited MSHR resources.

Machine-learning-based methods have also been widely used to learn complex relationships among programs, hardware characteristics, and performance outcomes. Ardalani et al. [**?**] proposed XAPP, which predicts GPU execution time from features of a single-threaded CPU implementation before GPU porting. Zheng et al. [**?**] introduced LACross, which predicts phase-level cross-platform performance and power traces from hardware-counter statistics collected on a host platform. Wang et al. [**?**] used deep neural networks to predict benchmark performance for new CPUs and workloads from public CPU specifications and benchmark datasets.

REFERENCES

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.